Funções SQL e o projeto Loja Virtual

Banco de Dados 2 · Aula 13 · 03/11/2026

SQL: Structured Query Language

Correção da prova 1

As 3 consultas que a turma menos soube explicar

1º passo

A pergunta

O que o enunciado quer saber? Diga com as suas palavras.

2º passo

Tabelas e JOIN

Onde estão os dados? Como as tabelas se ligam?

3º passo

O filtro

Quais linhas entram? Quais ficam de fora?

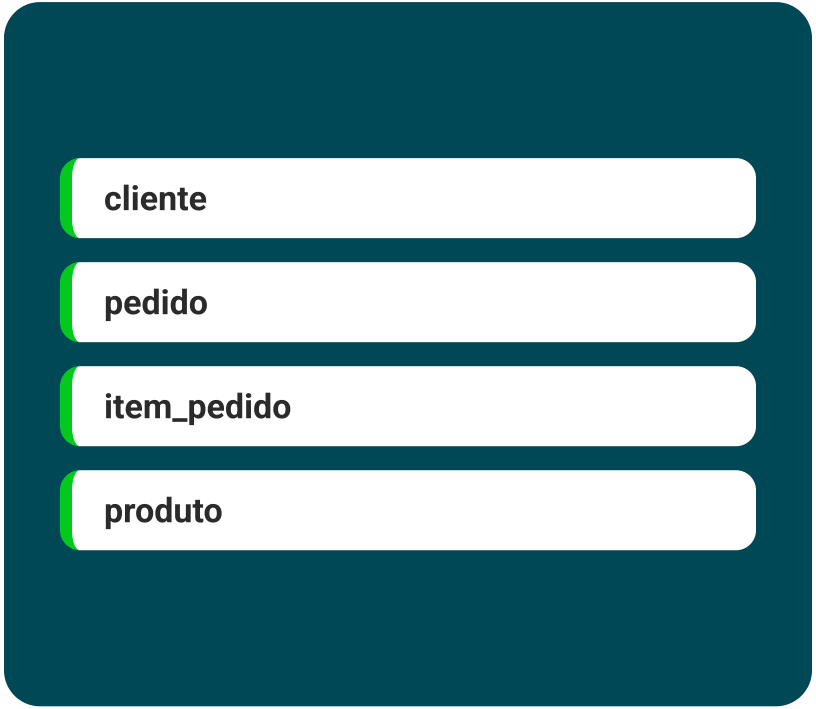
Um projeto só para a turma

A Loja Virtual, feita em duplas

Daqui para a frente, todos trabalham no mesmo banco. As duplas rodam a mesma consulta e comparam o resultado: tem de ser igual.

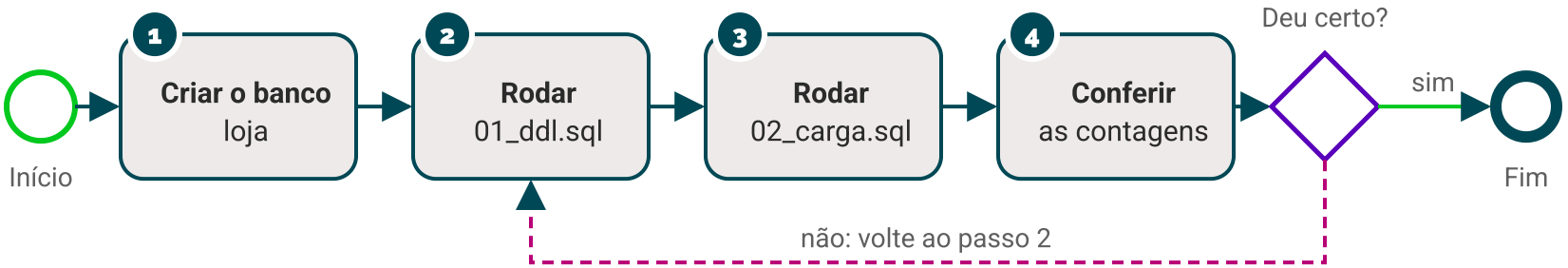
1
projeto para toda a turma

2
alunos por dupla



Montando o banco da dupla

Siga os passos em ordem, no cliente de banco de dados que preferir: DBeaver, DataGrip ou outro



Esperado: 120 pedidos, 40 clientes e 30 produtos. DDL: Data Definition Language. Para rodar o script inteiro no DBeaver, use Alt+X.

Texto e número

Cada função responde a uma pergunta da loja

LPAD

Qual é o número do pedido?

Completa com zeros à esquerda.

INITCAP · TRIM

Como limpar um nome?

Tira espaços e acerta as maiúsculas.

SUBSTRING

Como esconder o CPF?

Pega só um pedaço do texto.

REPLACE

Como criar um login?

Troca um trecho do texto por outro.

ROUND

Qual é o ticket médio?

Arredonda para as casas que você pedir.

Antes da função, a pergunta.

Sempre diga o que a loja quer saber.

LPAD: zeros à esquerda

Como mostrar o número do pedido como PD-2026-000001?

```sql
SELECT id,   -- escolhe as colunas; id é o número do pedido
  'PD-2026-' ||   -- texto fixo; || junta com o próximo item
  LPAD(id::text, 6, '0')   -- completa o id com zeros à esquerda
    AS numero   -- AS: dá o nome «numero» à coluna do resultado
FROM pedido   -- FROM: a tabela de onde saem as linhas
LIMIT 5;   -- LIMIT: mostra só as 5 primeiras linhas
```

> Resultado: PD-2026-000001, PD-2026-000002... O ano 2026 está fixo só neste exemplo. O LPAD é explicado no próximo slide.

Dentro do LPAD

A função recebe 3 informações, nesta ordem

```
LPAD(   -- Left PAD: completa à esquerda
  id::text,   -- 1º: o texto; ::text transforma o número em texto
  6,   -- 2º: o tamanho final do resultado
  '0'   -- 3º: com o que completar o que falta
)   -- fecha a função
```

> Exemplo: LPAD('7', 6, '0') dá 000007. Se o texto for maior que o tamanho, o LPAD corta o que passar: LPAD('1234567', 6, '0') dá 123456.

TRIM e INITCAP: nome limpo

Como arrumar um nome digitado com espaços e letras misturadas?

```
SELECT INITCAP(   -- INITCAP: 1ª letra de cada palavra em maiúscula
  TRIM('   mARIA sILVA  ')   -- TRIM: tira os espaços das pontas
) AS limpo;   -- AS: nome da coluna; resultado «Maria Silva»
```

> Uma função dentro da outra: a de dentro roda primeiro. O TRIM limpa as pontas e o INITCAP arruma as maiúsculas.

REPLACE: trocar um trecho

Como criar um login a partir do nome do cliente?

```sql
SELECT REPLACE(   -- troca um trecho do texto por outro
    LOWER(nome),   -- 1º: o texto; LOWER deixa tudo em minúsculas
    ' ',   -- 2º: o trecho a procurar (um espaço)
    '_'   -- 3º: o que entra no lugar (um sublinhado)
  ) AS login   -- AS: nome da coluna; maria silva vira maria_silva
FROM cliente LIMIT 5;   -- tabela cliente; só 5 linhas
```

REPLACE(texto, de, para) troca todas as ocorrências de de por para.

SUBSTRING: um pedaço do texto

Como mascarar o CPF (Cadastro de Pessoas Físicas)?

```sql
SELECT  -- escolhe as colunas do resultado
  SUBSTRING(cpf FROM 1 FOR 3)  -- 3 dígitos, do 1º
  || '.***.***-'  -- || junta; o texto fixo esconde o meio
  || SUBSTRING(cpf FROM 10 FOR 2)  -- 2 dígitos, do 10º
  AS cpf_mascarado  -- nome da coluna: 111.***.***-19
FROM cliente LIMIT 5;  -- tabela cliente; só 5 linhas
```

FROM é a posição inicial e FOR é quantos caracteres pegar.

ROUND: arredondar

Qual é o ticket médio dos pedidos que viraram venda?

```sql
SELECT ROUND(   -- ROUND(número, casas): arredonda
    AVG(total),   -- 1º: AVG calcula a média do total
    2   -- 2º: quantas casas decimais manter
  ) AS ticket_medio   -- AS: nome da coluna
FROM pedido   -- tabela de onde vêm os totais
WHERE status IN   -- WHERE: só pedidos que viraram venda
  ('pago', 'enviado', 'entregue');   -- status aceitos
```

Exemplo: ROUND(71.866, 2) dá 71.87.

Data, hora e condição

De novo, uma pergunta da loja para cada função

INTERVAL

Qual é o prazo para pagar?

Soma um tempo a uma data.

AGE

Quanto demorou a entrega?

Tempo entre duas datas.

DATE_TRUNC

Quantos pedidos por mês?

Volta a data ao início do mês.

EXTRACT

Qual dia da semana vende mais?

Tira um pedaço da data.

CASE

O pedido é pequeno, médio ou grande?

Escolhe um valor por condição.

COALESCE

Qual cupom? Ou «sem cupom»?

Troca o nulo por outro valor.

INTERVAL: somar um tempo

Qual é o prazo para pagar, 48 horas depois de fechar o pedido?

```sql
SELECT numero,   -- o número do pedido
  fechado_em   -- quando o pedido foi fechado
    + INTERVAL '48 hours'   -- soma 48 horas à data e hora
    AS prazo   -- AS: nome da coluna
FROM pedido   -- tabela de origem
WHERE status =   -- WHERE: filtra pelo status
  'aguardando_pagamento';   -- ainda sem pagamento
```

Data e hora + INTERVAL dá outra data e hora; data e hora − data e hora dá um intervalo.

AGE: tempo entre duas datas

Quanto tempo levou do pedido até a entrega?

```sql
SELECT numero,   -- o número do pedido
  AGE(   -- AGE(fim, início): tempo entre duas datas
    entregue_em,   -- 1º: quando foi entregue (o fim)
    criado_em   -- 2º: quando o pedido foi criado (o início)
  ) AS ate_a_entrega   -- AS: nome da coluna
FROM pedido   -- tabela de origem
WHERE status = 'entregue';   -- só pedidos já entregues
```

AGE devolve um intervalo, por exemplo 3 days 04:10:00.

DATE_TRUNC: agrupar por mês

Quantos pedidos houve em cada mês?

```sql
SELECT  -- escolhe as colunas do resultado
  DATE_TRUNC('month', criado_em)::date  -- dia 1 do mês
    AS mes,  -- AS: nome da coluna
  COUNT(*) AS pedidos  -- COUNT(*): conta as linhas de cada mês
FROM pedido  -- tabela de origem
GROUP BY 1  -- agrupa pela 1ª coluna (o mês)
ORDER BY 1;  -- ordena pela 1ª coluna: do mês mais antigo
```

DATE_TRUNC('month', d) volta a data ao dia 1 do mês; o ::date mostra só a data.

EXTRACT: um pedaço da data

Qual dia da semana tem mais vendas?

```sql
SELECT   -- escolhe as colunas do resultado
  EXTRACT(DOW FROM criado_em)   -- dia da semana (0 = domingo)
    AS dia,   -- AS: nome da coluna
  COUNT(*) AS pedidos   -- quantos pedidos em cada dia
FROM pedido   -- tabela de origem
GROUP BY 1   -- agrupa pela 1ª coluna (o dia)
ORDER BY 2 DESC;   -- do dia com mais pedidos ao com menos
```

> DOW: Day Of Week. No pedido, tire também carrinhos e cancelados.

CASE: escolher por condição

O pedido é pequeno (até 150), médio (até 500) ou grande?

```sql
SELECT CASE  -- CASE: escolhe um valor conforme a condição
  WHEN total < 150 THEN 'pequeno'  -- menor que 150: pequeno
  WHEN total < 500 THEN 'médio'  -- menor que 500: médio
  ELSE 'grande'  -- todo o resto: grande
END AS faixa  -- AS: nome da coluna
FROM pedido;  -- tabela de origem
```

O CASE testa de cima para baixo e fica com a primeira condição verdadeira. Para contar por faixa, use COUNT(*) e GROUP BY.

COALESCE: trocar o nulo

Qual cupom o pedido usou? Se não usou, mostre «sem cupom». (1 de 2)

```sql
SELECT p.numero,   -- p.numero: número do pedido
  COALESCE(   -- devolve o 1º valor que não é nulo
    c.codigo,   -- 1º: código do cupom (nulo sem cupom)
    'sem cupom'   -- 2º: usado quando o 1º é nulo
  ) AS cupom   -- AS: nome da coluna
```

NULL quer dizer «sem valor». O COALESCE olha os valores em ordem e fica com o primeiro que existe.

COALESCE: de onde vem o cupom

O resto da consulta junta o pedido ao cupom (2 de 2)

```
FROM pedido p   -- p: apelido de pedido
LEFT JOIN cupom c   -- c: apelido; mantém pedidos sem cupom
  ON c.id = p.cupom_id   -- liga o cupom ao pedido
WHERE p.status = 'entregue'   -- só pedidos entregues
LIMIT 8;   -- só 8 linhas
```

> Sem o LEFT JOIN, os pedidos sem cupom sumiriam do resultado; com ele, o cupom vem nulo e o COALESCE troca por «sem cupom».

O que é uma view

Uma consulta guardada com um nome

1 · Escreva

O SELECT que você já sabe

Teste até o resultado ficar certo.

2 · Guarde

CREATE VIEW nome AS

Coloque essas palavras na frente do SELECT.

3 · Use

SELECT * FROM nome

A view se usa como se fosse uma tabela.

A view não guarda dados. Cada vez que você a usa, o SELECT roda de novo e mostra os dados de agora.

CREATE VIEW: guardando a consulta

A consulta do LPAD, agora com nome

```sql
CREATE OR REPLACE VIEW vw_numero_pedido AS   -- cria a view; se j
SELECT id,   -- daqui para baixo é o mesmo SELECT do LPAD
  'PD-2026-' ||   -- texto fixo; || junta com o próximo item
  LPAD(id::text, 6, '0')   -- completa o id com zeros à esquerda
    AS numero   -- o nome da coluna no resultado
FROM pedido;   -- sem LIMIT: a view guarda a pergunta inteira
```

O nome da view começa com vw_ para você reconhecer que não é uma tabela. Rodar esse comando não mostra linhas: ele só guarda a consulta.

Usando a view

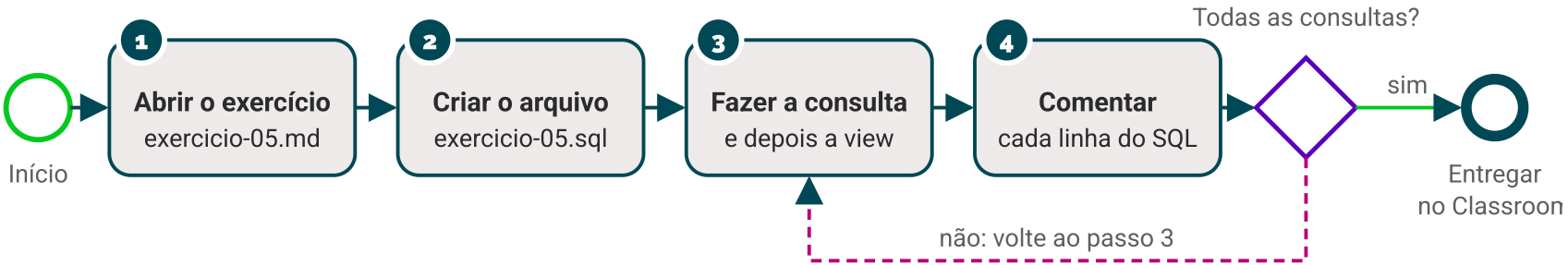
Agora ela entra no FROM, no lugar da tabela

```sql
SELECT *   -- * traz todas as colunas da view
FROM vw_numero_pedido   -- FROM: a «tabela» agora é a view
WHERE id ⩽ 120   -- filtros funcionam como numa tabela
ORDER BY id   -- a ordem também: aqui, pelo número do pedido
LIMIT 5;   -- o LIMIT fica aqui, em quem usa a view
```

> Para apagar a view: DROP VIEW vw_numero_pedido;. Para mudar, rode o CREATE OR REPLACE VIEW de novo.

Exercício 5: funções SQL e view simples

Pode fazer em dupla. Cada aluno entrega o seu, no Classroom



> Escreva um comentário (--) em cada linha do SQL, dizendo o que ela faz. Isso ajuda você a estudar.
> SQL: Structured Query Language.

Consulta 1: nome e código dos produtos

A loja quer etiquetas: nome em maiúsculas e o código sem o «SKU-»

```sql
SELECT UPPER('Teclado Plus');   -- UPPER: deixa o texto em maiúsculas
SELECT REPLACE('SKU-0001', 'SKU-', '');   -- REPLACE: troca «SKU-» por nada
```

> Teste cada função com um valor solto antes de usar nas colunas. SKU: Stock Keeping Unit, o código interno do produto.

Consulta 1: monte a consulta

Preencha os espaços. Todos os produtos, sem LIMIT

```
SELECT ___ AS produto,   -- 1ª coluna: o nome em maiúsculas (UPPER da c
       ___ AS codigo     -- 2ª coluna: o sku sem o prefixo (REPLACE da col
FROM ___   -- a tabela onde estão os produtos
ORDER BY ___;   -- a ordem: o id, para a lista sair sempre igual
```

> Confira: o resultado tem 30 linhas, e a primeira é TECLADO PLUS com o código 0001.

Consulta 1: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```sql
CREATE OR REPLACE VIEW vw_produto_codigo AS   -- guarda a consul
___;   -- cole aqui a consulta 1, sem o ponto e vírgula do fim
SELECT * FROM vw_produto_codigo;   -- usa a view: tem que dar o mesm
```

> Se o resultado da view for igual ao da consulta, a consulta 1 está pronta. Escreva um comentário em cada linha.

Consulta 2: login de cada cliente

Quero o que vem antes do @ no e-mail de cada cliente

```
1 SELECT POSITION('@' IN 'ana@exemplo.test');   -- POSITION: em qu
2 SELECT SUBSTRING('ana@exemplo.test' FROM 1 FOR 3);   -- SUBSTR
```

> O login tem posição do @ − 1 letras. No exemplo: 4 − 1 = 3 letras, ana.

Consulta 2: monte a consulta

Preencha os espaços. Todos os clientes, sem LIMIT

```
SELECT nome,   -- a primeira coluna é o nome do cliente
       SUBSTRING(email FROM 1 FOR ___) AS usuario   -- letras a
FROM ___   -- a tabela dos clientes
ORDER BY ___;   -- a ordem: o id
```

Confira: o resultado tem 40 linhas, e a primeira é Cliente 01 com o usuário cliente01.

Consulta 2: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```
CREATE OR REPLACE VIEW vw_cliente_usuario AS   -- guarda a consu
___;   -- cole aqui a consulta 2, sem o ponto e vírgula do fim
SELECT * FROM vw_cliente_usuario;   -- usa a view: tem que dar o mes
```

> Se o resultado da view for igual ao da consulta, a consulta 2 está pronta. Escreva um comentário em cada linha.

Consulta 3: CEP no formato certo

O endereço de entrega com o CEP escrito como 01000-311

```sql
SELECT LEFT('01000311', 5);   -- LEFT: as 5 primeiras letras → 01000
SELECT RIGHT('01000311', 3);   -- RIGHT: as 3 últimas letras → 311
SELECT '01000' || '-' || '311';   -- ||: junta os textos → 01000-311
```

> CEP: Código de Endereçamento Postal. UF: Unidade Federativa, o estado.

Consulta 3: monte a consulta

Preencha os espaços. Todos os endereços, sem LIMIT

```sql
SELECT cidade, uf,   -- cidade e estado do endereço
       ___ AS cep   -- LEFT 5 letras, o traço, RIGHT 3 letras (junte com
FROM ___   -- a tabela dos endereços
ORDER BY ___;   -- a ordem: o id
```

> Confira: o resultado tem 60 linhas, e a primeira é Rio de Janeiro, RJ, 01000-311.

Consulta 3: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```sql
CREATE OR REPLACE VIEW vw_endereco_cep AS   -- guarda a consulta
___;   -- cole aqui a consulta 3, sem o ponto e vírgula do fim
SELECT * FROM vw_endereco_cep;   -- usa a view: tem que dar o mesmo r
```

> Se o resultado da view for igual ao da consulta, a consulta 3 está pronta. Escreva um comentário em cada linha.

Consulta 4: parcelas em 3 vezes

Quanto fica cada parcela se o cliente parcelar o pagamento em 3 vezes

```sql
SELECT ROUND(100 / 3.0, 2);   -- ROUND: arredonda para 2 casas → 33.33
SELECT CEIL(100 / 3.0);   -- CEIL: arredonda para cima, em inteiro → 34
```

> ROUND escolhe as casas. CEIL sempre sobe para o próximo inteiro (e FLOOR sempre desce).

Consulta 4: monte a consulta

Preencha os espaços. Todos os pagamentos, sem LIMIT

```sql
SELECT pedido_id, valor,   -- o pedido e o valor pago
       ___ AS parcela,   -- valor dividido por 3, com 2 casas (ROUND)
       ___ AS parcela_inteira   -- valor dividido por 3, para cima (C
FROM ___   -- a tabela dos pagamentos
ORDER BY ___;   -- a ordem: o id
```

Confira: o resultado tem 90 linhas, e a primeira é o pedido 31, valor 215.60, parcela 71.87 e 72.

Consulta 4: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```sql
CREATE OR REPLACE VIEW vw_pagamento_parcelas AS   -- guarda a c
___;   -- cole aqui a consulta 4, sem o ponto e vírgula do fim
SELECT * FROM vw_pagamento_parcelas;   -- usa a view: tem que dar c
```

> Se o resultado da view for igual ao da consulta, a consulta 4 está pronta. Escreva um comentário em cada linha.

Consulta 5: situação dos cupons

Quantos dias faltam para cada cupom vencer e qual é a situação dele

```
1 SELECT (CURRENT_DATE + 10) - CURRENT_DATE;   -- data menos data =
2 SELECT CASE WHEN 5 < 3 THEN 'primeiro' WHEN 5 < 9 THEN 'seg
```

O CASE para na primeira condição verdadeira. Por isso a ordem das condições importa.

Consulta 5: monte a consulta

Preencha os espaços. Todos os cupons

```sql
SELECT codigo, valido_ate,  -- o cupom e a validade
       valido_ate - CURRENT_DATE AS dias,  -- dias até vencer (n
       CASE WHEN ___ THEN 'vencido'  -- 1ª condição: a validade já
            WHEN ___ THEN 'vence em até 30 dias'  -- 2ª condiç
            ELSE 'válido' END AS situacao  -- todo o resto
FROM ___  -- a tabela dos cupons
ORDER BY ___;  -- do que vence primeiro ao que vence por último
```

Confira: uma linha para cada cupom; VENCIDO15 é o único vencido, e NATAL20 e BEMVINDO10 estão válido.

Consulta 5: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```sql
CREATE OR REPLACE VIEW vw_cupom_situacao AS   -- guarda a consul
___;   -- cole aqui a consulta 5, sem o ponto e vírgula do fim
SELECT * FROM vw_cupom_situacao;   -- usa a view: tem que dar o mesm
```

> Se o resultado da view for igual ao da consulta, a consulta 5 está pronta. Escreva um comentário em cada linha.

Consulta 6: avaliações em texto

A nota em palavras e o comentário, mesmo quando não há comentário

```sql
SELECT COALESCE(NULL, 'sem comentário');   -- COALESCE: troca o nu
SELECT COALESCE('Ótimo!', 'sem comentário');   -- se não for nul
```

> NULL é «sem valor». Com COALESCE você escolhe o que mostrar no lugar.

Consulta 6: monte a consulta

Preencha os espaços. Todas as avaliações, sem LIMIT

```sql
SELECT produto_id, nota,   -- o produto avaliado e a nota
       CASE WHEN ___ THEN 'ruim'   -- 1ª condição: nota 1 ou 2
            WHEN ___ THEN 'regular'   -- 2ª condição: nota 3
            ELSE 'boa' END AS avaliacao,   -- todo o resto: 4 e 5
       COALESCE(___, 'sem comentário') AS comentario   -- o c
FROM ___   -- a tabela das avaliações
ORDER BY ___;   -- a ordem: o id
```

Confira: o resultado tem 50 linhas, e a primeira é o produto 6, nota 2, ruim.

Consulta 6: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```
CREATE OR REPLACE VIEW vw_avaliacao_texto AS   -- guarda a consu
___;   -- cole aqui a consulta 6, sem o ponto e vírgula do fim
SELECT * FROM vw_avaliacao_texto;   -- usa a view: tem que dar o mes
```

> Se o resultado da view for igual ao da consulta, a consulta 6 está pronta. Escreva um comentário em cada linha.

Consulta 7: total recebido por mês

Quanto a loja recebeu em cada mês

```
1 SELECT DATE_TRUNC('month', CURRENT_DATE);   -- DATE_TRUNC: corta
2 SELECT TO_CHAR(CURRENT_DATE, 'MM/YYYY');   -- TO_CHAR: escreve a
```

SUM e COUNT juntam várias linhas em uma. O GROUP BY diz de quais linhas formar cada grupo.

Consulta 7: monte a consulta

Preencha os espaços. Um resultado por mês

```sql
SELECT TO_CHAR(DATE_TRUNC('month', pago_em), 'MM/YYYY') AS
       ___ AS pagamentos,   -- quantos pagamentos no mês (COUNT)
       ___ AS total   -- a soma dos valores do mês (SUM)
FROM ___   -- a tabela dos pagamentos
GROUP BY ___   -- um grupo por mês: DATE_TRUNC('month', pago_em)
ORDER BY ___;   -- em ordem de calendário: DATE_TRUNC('month', pago_em)
```

> Confira: a soma da coluna total é igual a SELECT SUM(valor) FROM pagamento; e a soma de pagamentos é 90.

Consulta 7: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```sql
CREATE OR REPLACE VIEW vw_recebido_por_mes AS   -- guarda a cons
___;   -- cole aqui a consulta 7, sem o ponto e vírgula do fim
SELECT * FROM vw_recebido_por_mes;   -- usa a view: tem que dar o m
```

> Se o resultado da view for igual ao da consulta, a consulta 7 está pronta. Escreva um comentário em cada linha.

Consulta 8: carrinhos parados

Há quantas horas cada carrinho está parado

```
1 SELECT LOCALTIMESTAMP - INTERVAL '5 hours';   -- agora menos 5 h
2 SELECT EXTRACT(EPOCH FROM INTERVAL '2 hours');   -- EPOCH: o in
```

> Um intervalo é um tempo, como «2 horas». EPOCH o transforma em segundos; dividindo por 3600 você tem horas.

Consulta 8: monte a consulta

Preencha os espaços. Só os carrinhos

```sql
SELECT id, criado_em,   -- o carrinho e quando ele foi criado
       ROUND(___ / 3600) AS horas   -- EXTRACT(EPOCH FROM agora - cr
FROM ___   -- carrinho é um pedido
WHERE ___   -- só o status 'carrinho'
ORDER BY ___;   -- o mais antigo primeiro
```

Confira: o resultado tem 10 carrinhos, e o primeiro é o mais antigo.

Consulta 8: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```
CREATE OR REPLACE VIEW vw_carrinho_parado AS   -- guarda a consu
___;   -- cole aqui a consulta 8, sem o ponto e vírgula do fim
SELECT * FROM vw_carrinho_parado;   -- usa a view: tem que dar o mes
```

> Se o resultado da view for igual ao da consulta, a consulta 8 está pronta. Escreva um comentário em cada linha.

Consulta 9: histórico de preços

Cada preço de cada produto, com a data e uma promoção de 10%

```sql
SELECT 'R$ ' || TO_CHAR(28.9, 'FM999990.00');   -- número com 2
SELECT TO_CHAR(CURRENT_DATE, 'DD/MM/YYYY');   -- a data como dia,
SELECT ROUND(28.9 * 0.9, 2);   -- 10% de desconto = vezes 0,9 → 26.01
```

O JOIN liga cada preço ao seu produto pelo id. pp e pr são apelidos das tabelas.

Consulta 9: monte a consulta

Preencha os espaços. Todos os preços de todos os produtos

```sql
SELECT pr.nome AS produto,   -- o nome vem da tabela produto (apelido
       ___ AS preco,   -- 'R$ ' || TO_CHAR(preco, 'FM999990.00')
       ___ AS desde,   -- TO_CHAR da vigencia_inicio como DD/MM/YYYY
       ___ AS preco_promocional   -- ROUND do preço vezes 0.9, com 2
FROM preco_produto pp   -- os preços (apelido pp)
JOIN ___ pr ON ___   -- liga o preço ao produto pelo id
ORDER BY ___;   -- por produto e, depois, pela data do preço
```

Confira: o resultado tem 37 linhas, e a primeira é Teclado Plus, R$ 141.90, 01/01/2025, 127.71.

Consulta 9: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```sql
CREATE OR REPLACE VIEW vw_preco_historico AS   -- guarda a consu
___;   -- cole aqui a consulta 9, sem o ponto e vírgula do fim
SELECT * FROM vw_preco_historico;   -- usa a view: tem que dar o mes
```

> Se o resultado da view for igual ao da consulta, a consulta 9 está pronta. Escreva um comentário em cada linha.

Consulta 10: percentual de desconto

Quanto do pedido foi desconto, nos pedidos que usaram cupom

```sql
SELECT ROUND(10 * 100.0 / 200, 1);   -- 10 é quantos por cento de 2
SELECT 10 / NULLIF(0, 0);   -- NULLIF: devolve nulo quando os dois são
```

Dividir por zero dá erro. NULLIF(subtotal, 0) troca o zero por nulo, e a conta devolve nulo.

Consulta 10: monte a consulta

Preencha os espaços. Só os pedidos que usaram cupom

```sql
SELECT id, subtotal, desconto,  -- o pedido, o subtotal e o desconto
       ROUND(desconto * 100.0 / ___, 1) AS percentual  -- NU
FROM ___   -- a tabela dos pedidos
WHERE ___   -- só quem usou cupom: cupom_id não é nulo
ORDER BY ___;   -- a ordem: o id
```

Confira: o resultado tem 21 pedidos, e o primeiro é o 7, com percentual 10.0.

Consulta 10: a mesma consulta como view

Logo depois da consulta, guarde-a com nome e use a view

```sql
CREATE OR REPLACE VIEW vw_desconto_pedido AS   -- guarda a consu
___;   -- cole aqui a consulta 10, sem o ponto e vírgula do fim
SELECT * FROM vw_desconto_pedido;   -- usa a view: tem que dar o mes
```

> Se o resultado da view for igual ao da consulta, a consulta 10 está pronta. Escreva um comentário em cada linha.